METR4202 Final Documentation

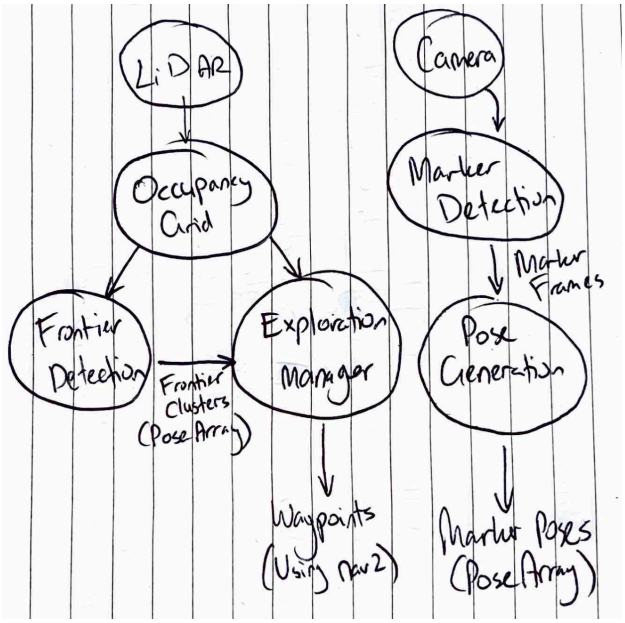
Team 29 - Michael Vossen, Mitchell Taylor, Thomas Oram, Robin Grace

Full System Description

The goal of the Turtlebot is to explore the entirety of the map, whilst documenting the locations of any ArUco markers that it observes in its movements. The LiDAR sensors onboard the robot are used to implement SLAM, allowing localisation and mapping to occur simultaneously as it navigates.

The exploration of the robot is guided by a Frontier Detection node, which calculates a collection of potentially viable frontiers whenever the Occupancy Grid topic is updated. Frontiers are selected based on the concentration of unknown cells, and the density and thresholds for a viable frontier have been experimentally optimised.These locations are used by the Exploration Manager node to send waypoints to the Turtlebot with the goal of observing as many of the frontier clusters as possible. In this way the Turtlebot is able to explore the entirety of a generalised map.

While the Turtlebot is navigating through the environment, a Marker Detection node stores the current camera feed and searches for ArUco markers on a fixed interval. It makes use of OpenCV to identify the type of marker and to determine its location relative to the camera. The node maintains a local memory of all markers that have been identified and their locations to prevent duplicate markers from overwriting each other.



Component Descriptions

Frontier Detection

The frontier detector was implemented as a ROS 2 Python node that converts the occupancy grid produced by SLAM into candidate exploration locations. The node subscribes to the `/map` topic using the `nav_msgs/OccupancyGrid` message type. It processes each received map and publishes detected frontier cells, representative frontier positions and navigation candidates.

The algorithm consists of three main stages: initial frontier detection, clustering and filtering of frontier cells, and selection of representative frontiers. These stages progressively reduce the number of potential exploration points to a smaller set of suitable navigation candidates.

An occupancy-grid cell is classified as a frontier when:

1. The cell is known to be free (0).

2. At least three cells in its eight-connected neighbourhood are unknown (-1).

The neighbourhood is clipped at the map boundaries so cells along the edge of the occupancy grid can also be processed. The threshold of three unknown neighbours was chosen to reduce weak or isolated frontier detections.

The resulting frontier cells are divided into connected components using an eight-connected flood-fill search. Components containing fewer than 4 cells are discarded. This prevents small or isolated frontier groups from being passed to the exploration system.

To reduce the number of navigation candidates, each remaining frontier cell is assigned a density score equal to the number of frontier cells within a 0.25 m radius. Cells are considered in descending score order, prioritising those surrounded by more frontier cells.

Before selecting a candidate, the node checks that it has sufficient clearance from nearby obstacles. Candidates within 0.3 m of an occupied cell (occupancy value greater than 50) are rejected. A candidate is then retained only when it is at least 0.3 m from every previously selected representative. This produces representative points in dense frontier regions while reducing redundant navigation goals and avoiding points too close to obstacles.

Grid coordinates are converted to world coordinates using the map resolution, cell-centre offset, map-origin translation and map-origin yaw rotation. This ensures the published points remain correct even when the occupancy-grid origin is rotated.

The node publishes:

- `/frontier_cells`: Filtered frontier cells as red RViz Marker points.

- `/frontier_representatives`: Selected representatives as green RViz Marker points.

- `/frontier_candidates`: The same representatives as a `geometry_msgs/PoseArray` for the exploration manager.

Empty candidate arrays are also published when no valid representatives are found, allowing the exploration manager to identify when no exploration candidates are currently available.

Exploration Manager

The Exploration Manager was implemented as a ROS 2 Python node responsible for selecting suitable frontier locations and sending navigation goals to the TurtleBot using Nav2. It subscribes to /frontier_candidates, published by the Frontier Detection node, and /map, which provides the current nav_msgs/OccupancyGrid. It also uses TF2 to obtain the robot pose from the transformation between the map and base_link frames.

Each frontier candidate is first checked to ensure that it is suitable for navigation. Candidates closer than 0.30 m to the robot are ignored to prevent very short and unnecessary navigation goals. Frontiers located within 0.40 m of a previously failed navigation goal are also excluded, reducing repeated attempts to reach slightly shifted versions of the same inaccessible frontier. Previously reached frontier coordinates are also stored so that the same exact location is not selected again.

The remaining frontier candidates are evaluated using three factors: distance from the robot, required heading change, and the amount of unexplored map surrounding the candidate. Euclidean distance is calculated between the robot and each candidate. The required rotation is calculated from the difference between the robot's current heading and the direction of the frontier. Information gain is estimated by counting unknown occupancy-grid cells within a 1.0 m radius around each frontier.

The Exploration Manager also gives preference to frontiers requiring no more than a 90° initial turn. If at least one such frontier is available, candidates requiring larger turns are temporarily excluded. However, if no forward-preferred candidates are available, the manager allows larger turns so that exploration can continue rather than becoming stuck. The information gain and distance values are normalised before being combined with the heading change using the weighted scoring function:

$$score = 0.30(normalisedinformationgain) - 0.40(normaliseddistance) - 0.30(normalisedheadingchange$$

This gives the greatest weighting to shorter travel distance, while still favouring frontiers expected to reveal a large amount of unexplored space and penalising unnecessary rotation. The highest-scoring candidate is then selected as the next exploration goal. The selected frontier is sent to Nav2 using the NavigateToPose action. Before sending the goal, the Exploration Manager calculates the goal orientation so that the robot faces in the direction of travel toward the frontier. Nav2 is then responsible for path planning, obstacle avoidance, recovery behaviours and determining whether the navigation attempt succeeds or fails.

When Nav2 reports that a frontier has been successfully reached, its position is stored and the Exploration Manager enters a WAITING_FOR_UPDATE state. It waits for a new /frontier_candidates message before selecting another destination, ensuring that the next decision uses updated map and frontier information. If navigation fails, the frontier location is stored and nearby candidates within 0.40 m are excluded from future selection.

Empty frontier arrays are also monitored. After three consecutive empty frontier updates, the state is changed to EXPLORATION_COMPLETE, indicating that no further unexplored frontiers are currently available.

Aruco Marker Detection

This component makes use of the following existing libraries:

- cv2. Capable of detecting ArUco markers within a 2D image and estimating their poses relative to the camera frame. It does this for a specific family of ArUco markers, and in this case it has been configured for a dictionary of 250 6x6 marker types.
- cv2Bridge
- tf2_ros, specifically the TransformListener and Buffer class, as well as the methods that calculate the transformation from one frame to another. Used to convert marker's positions in the camera frame into the global map frame.

The ROS2 Python node is initialised with the appropriate ArUco parameters that denote the family of markers and their size. Additionally subscriptions are set up for /camera/camera_info and /camera/image_raw topics, which contain the sensor_msgs message types CameraInfo and Image respectively.

The callback function for the CameraInfo subscription reads the message and uses the information on the camera to initialise internal variables representing the image header (timestamp and frame data), the camera matrix and the camera distortion. This is assumed to be static throughout the operation of the turtlebot and so the subscription is ended after the first InfoMsg is received

The node updates a local copy of the turtlebot's camera feed, which is processed every 0.5 seconds to improve performance and to prevent operations from stalling. The raw image is converted to a cv2 compatible form using cvBridge and any ArUco markers in the image are detected using the OpenCV library. It makes of a Perspective-n-Point solver algorithm to estimate the 3D pose of each marker relative to the camera frame.

This pose is transformed into the global /map frame which serves as the static frame for the TurtleBot. This is performed using the tf2_geometry_msg library which uses the TF Transformation Tree to calculate the transform between the camera frame and the map frame. By obtaining the global Pose of the ArUco marker, we are able to compare it to the previously identified markers and determine if it is a repeat detection. If so, the location is updated to be the average of the two positions, which aids the localisation of the marker's true position.

Each Pose is converted into a MarkerArray which is used for visualisation. In this way the Turtlebot is able to log and display the positions of every ArUco marker it comes across, updating the visualisation every 1 second.